

Importing and Exporting data

Feb 12, 2025

Today

- `read.table()` and subfunctions
- `write.table()` and `write.csv()`
- `readLines()`
- `writeLines()`
- more about `save()`

read.table()

A lot of options...

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
           dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
           row.names, col.names, as.is = !stringsAsFactors,
           na.strings = "NA", colClasses = NA, nrows = -1,
           skip = 0, check.names = TRUE, fill = !blank.lines.skip,
           strip.white = FALSE, blank.lines.skip = TRUE,
           comment.char = "#",
           allowEscapes = FALSE, flush = FALSE,
           stringsAsFactors = FALSE,
           fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)
```

- **file** = file name you want to read in as a character (in quotes)
- **header** = does it have a header? (T or F)
- **sep** = what symbol is separating the values in each column

3/25

read.table()

...more options...

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
           dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
           row.names, col.names, as.is = !stringsAsFactors,
           na.strings = "NA", colClasses = NA, nrows = -1,
           skip = 0, check.names = TRUE, fill = !blank.lines.skip,
           strip.white = FALSE, blank.lines.skip = TRUE,
           comment.char = "#",
           allowEscapes = FALSE, flush = FALSE,
           stringsAsFactors = FALSE,
           fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)
```

- **quote** = What characters are used to denote quotes?
- **dec** = What's used to denote decimals? Usually "." or ",".
- **row.names** = which column are the rownames in?
- **col.names** = specify column names directly

4/25

read.table()

...more options...

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
  dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
  row.names, col.names, as.is = !stringsAsFactors,
  na.strings = "NA", colClasses = NA, nrow = -1,
  skip = 0, check.names = TRUE, fill = !blank.lines.skip,
  strip.white = FALSE, blank.lines.skip = TRUE,
  comment.char = "#",
  allowEscapes = FALSE, flush = FALSE,
  stringsAsFactors = FALSE,
  fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)
```

- **as.is** = Which columns to keep strings as factors (prevent conversion)?
- **na.strings** = What character symbols should be interpreted as NAs?
- **colClasses** = A vector of classes to be assumed for columns

5/25

read.table()

...more options...

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
  dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
  row.names, col.names, as.is = !stringsAsFactors,
  na.strings = "NA", colClasses = NA, nrow = -1,
  skip = 0, check.names = TRUE, fill = !blank.lines.skip,
  strip.white = FALSE, blank.lines.skip = TRUE,
  comment.char = "#",
  allowEscapes = FALSE, flush = FALSE,
  stringsAsFactors = FALSE,
  fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)
```

- **nrow** = How many total rows to read in (after the header, if applicable)
- **skip** = How many rows to skip before start reading. This includes skipping the header.
- **check.names** = Makes sure names are valid column names. No special characters, etc.

6/25

read.table()

...more options...

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
  dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
  row.names, col.names, as.is = !stringsAsFactors,
  na.strings = "NA", colClasses = NA, nrows = -1,
  skip = 0, check.names = TRUE, fill = !blank.lines.skip,
  strip.white = FALSE, blank.lines.skip = TRUE,
  comment.char = "#",
  allowEscapes = FALSE, flush = FALSE,
  stringsAsFactors = FALSE,
  fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)
```

- **fill** = Should missing values be filled in?
- **strip.white** = Should spaces or tabs before or after values be removed?
- **blank.lines.skip** = Should blank rows be ignored?

7/25

Know the options for **read.table()**

If you know how they work, you will save time by avoiding typing, find-and-replacing, etc.

8/25

read.csv() and read.delim()

".csv" = comma-separated values

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",  
         dec = ".", fill = TRUE, comment.char = "", ...)
```

Tab-delimited values

```
read.delim(file, header = TRUE, sep = "\t", quote = "\"",  
           dec = ".", fill = TRUE, comment.char = "", ...)
```

9/25

Wrappers for read.table()

Access the *object* `read.csv`.

```
read.csv
```

```
## function (file, header = TRUE, sep = ",", quote = "\"", dec = ".",  
##      fill = TRUE, comment.char = "", ...)  
## read.table(file = file, header = header, sep = sep, quote = quote,  
##      dec = dec, fill = fill, comment.char = comment.char, ...)  
## <bytecode: 0x7f8c5982a8b0>  
## <environment: namespace:utils>
```

10/25

Read Puromycin data

List all files in the directory with the extension “.csv”

```
list.files(pattern=".csv")
```

```
## [1] "Puromycin_data.csv"
```

“The Puromycin data frame has 23 rows and 3 columns of the reaction velocity versus substrate concentration in an enzymatic reaction involving untreated cells or cells treated with Puromycin.”—R help

Written as a “.csv” file

```
"", "conc", "rate", "state"  
"1", 0.02, 76, "treated"  
"2", 0.02, 47, "treated"  
"3", 0.06, 97, "treated"  
"4", 0.06, 107, "treated"  
"5", 0.11, 123, "treated"  
"6", 0.11, 139, "treated"  
"7", 0.22, 159, "treated"  
"8", 0.22, 152, "treated"
```

11/25

Read Puromycin data

```
pur<-read.csv("Puromycin_data.csv", row.names=1)  
head(pur)
```

```
##   conc rate  state  
## 1 0.02   76 treated  
## 2 0.02   47 treated  
## 3 0.06   97 treated  
## 4 0.06  107 treated  
## 5 0.11  123 treated  
## 6 0.11  139 treated
```

12/25

Write Puromycin data

Change to *tab-delimited text* using `write.table()`

```
write.table(pur, file = "Puromycin_data.delim", row.names=F, sep= "\t")
```

- There is no `write.delim()` function in base R.

```
"conc"  "rate"  "state"
0.02    76     "treated"
0.02    47     "treated"
0.06    97     "treated"
0.06    107    "treated"
0.11    123    "treated"
0.11    139    "treated"
0.22    159    "treated"
0.22    152    "treated"
0.56    191    "treated"
0.56    201    "treated"
1.1 207 "treated"
1.1 200 "treated"
0.02    67     "untreated"
0.02    51     "untreated"
0.06    84     "untreated"
```

13/25

readLines()

What if we just downloaded a massive file with an unknown extension, but we think it's text.

```
file.info("usearch.otus.97.tax")
```

```
##              size isdir mode              mtime              ctime
## usearch.otus.97.tax 16315973 FALSE  644 2013-11-11 09:48:10 2025-02-11 15:49:26
##              atime uid gid  uname grname
## usearch.otus.97.tax 2025-02-11 15:49:26 501  20 hgreen  staff
```

14/25

readLines()

```
tax<-readLines("usearch.otus.97.tax", n=4)
head(tax)
```

```
## [1] "Cluster_ID\tTaxonomy\tRank\tCluster_Size\tMin_GDist\tAvg_GDist\tVote\tMin_rank\tTaxa_cc"
## [2] "Cluster0\tBacteria;Proteobacteria;Betaproteobacteria;Burkholderiales;Alcaligenaceae;Alc"
## [3] "Cluster1\tBacteria;Firmicutes;Clostridia;Clostridiales;Lachnospiraceae;Roseburia\tgenus"
## [4] "Cluster10\tBacteria;Actinobacteria;Actinobacteria;Bifidobacteriales;Bifidobacteriaceae;"
```

- `readLines()` can be used to quickly sample and explore unfamiliar data

15/25

cat() and writeLines()

- `cat()` = raw flexible output to file or console

```
month.abb[1:4]
```

```
## [1] "Jan" "Feb" "Mar" "Apr"
```

```
cat(month.abb[1:4])
```

```
## Jan Feb Mar Apr
```

16/25

cat() and writeLines()

- `writeLines()` = similar to `cat()`, but default `sep = "\n"`

```
month.abb[1:4]
```

```
## [1] "Jan" "Feb" "Mar" "Apr"
```

```
writeLines(month.abb[1:4])
```

```
## Jan
```

```
## Feb
```

```
## Mar
```

```
## Apr
```

- `cat()` is probably not what you want unless you specify `sep`.

17/25

Saving big files

Read in "tax" file:

```
system.time(tax<-read.delim("usearch.otus.97.tax"))  
dim(tax)  
object.size(tax)
```

```
##      user  system elapsed  
##    0.494    0.015    0.511  
## [1] 93488     11  
## 13883864 bytes
```

18/25

Save as R object with compression

```
save(tax, file = "TAX_data_GZIP.Rdata", compress="gzip")
```

```
list.files()
```

```
## [1] "custom.css" "Importing and Exporting data.pdf"
## [3] "Importing-Exporting-Data.html" "Importing-Exporting-Data.Rmd"
## [5] "Puromycin_data.csv" "Puromycin_data.delim"
## [7] "qPCR-data.Rdata" "qPCR-data.txt"
## [9] "TAX_data_GZIP.Rdata" "usearch.otus.97.tax"
```

19/25

Compressed vs Original

Compressed R object 10X smaller than original text file

```
file.info("usearch.otus.97.tax")[1] / 10^6 #Original text file
file.info("TAX_data_GZIP.Rdata")[1]/ 10^6 # Compressed R object
```

```
## size
## usearch.otus.97.tax 16.31597
## size
## TAX_data_GZIP.Rdata 1.223305
```

- Multiple compression methods
- Compression adds time for packing and unpacking

20/25

Summary

- Ensure columns have a single class of data only, i.e., either numeric, factor, or character. Get rid of values like "<5".
- Know the options of `read.table()` and related functions.
- Windows users: check out packages `RExcel`, `xlsx`, `XLConnect`, or `gdata`.

21/25

Try it Yourself

1. Import the `qPCR-data.txt` file into R removing the metadata at the top of the file.
2. Find the median "CT" of all wells.
3. Save the data frame as a compressed R object.

22/25

Try it Yourself

1. Import the `qPCR-data.txt` file into R removing the metadata at the top of the file.

I did it this way:

```
dat<-read.delim("qPCR-data.txt", skip=41, na.strings="Undetermined")
```

23/25

Try it Yourself

2. Find the median "CT" of all wells.

```
head(dat[, 1:10])
```

```
##      Well Well.Position  Omit Sample.Name Target.Name      Task Reporter Quencher
## 1      1             A1 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
## 2      2             A2 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
## 3      3             A3 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
## 4      4             A4 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
## 5      5             A5 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
## 6      6             A6 FALSE   Sample 1      HF183 STANDARD        FAM  NFQ-MGB
##      CT Quantity
## 1      NA        1
## 2      NA        1
## 3      NA        1
## 4 19.215        1
## 5 18.648        1
## 6 18.506        1
```

```
median(dat$CT, na.rm=T)
```

```
## [1] 29.234
```

24/25

3. Save the data frame as a compressed R object.

```
save(dat, file="qPCR-data.Rdata", compress = "gzip")  
list.files()
```

```
## [1] "custom.css" "Importing and Exporting data.pdf"  
## [3] "Importing-Exporting-Data.html" "Importing-Exporting-Data.Rmd"  
## [5] "Puromycin_data.csv" "Puromycin_data.delim"  
## [7] "qPCR-data.Rdata" "qPCR-data.txt"  
## [9] "TAX_data_GZIP.Rdata" "usearch.otus.97.tax"
```